

Travail pratique : Heuristique DSATUR

Simulation et optimisation (SIO)

Auteurs : Florian Duruz, Rémy Bleuer

Date : 11 octobre 2026

1. Introduction

L'objectif de ce labo est d'étudier et de détailler l'heuristique de coloration séquentielle **DSATUR**. Cette méthode permet de colorier les sommets d'un graphe simple et non orienté en utilisant des degrés de saturation. Nous présentons dans ce rapport notre pseudo-code, ainsi qu'une analyse des complexités.

On note $G = (V, E)$ le graphe simple et non orienté, avec $n = |V|$ sommets et $m = |E|$ arêtes. Une coloration compatible associe à chaque sommet une couleur entière, les couleurs étant consécutives à partir de 0. Deux sommets adjacents n'ont jamais la même couleur.

L'objectif est d'obtenir une complexité temporelle et spatiale en $O(n^2)$ dans le pire des cas.

2. Structures de données

Structure	Type / taille	Rôle
<code>voisins</code>	tableau de n listes	Listes d'adjacence de G (donnée)
<code>degre</code>	tableau d'entiers, taille n	Degré initial de chaque sommet
<code>saturation</code>	tableau d'entiers, taille n	Degré de saturation courant
<code>couleur</code>	tableau d'entiers, taille n	Couleur de chaque sommet (-1 = non colorié)
<code>couleur_interdite</code>	tableau de booléens, taille $n \times n$	<code>couleur_interdite[s][c] = VRAI</code> si un voisin colorié de <code>s</code> a la couleur <code>c</code>

Pourquoi n couleurs possibles ? On note $\Delta(G)$ le degré maximal des sommets du graphe G . En référence à ce qui a été vu en cours, les heuristiques de coloration séquentielle, dont DSATUR, utilisent au plus $\Delta(G) + 1$ couleurs, cette propriété étant vérifiée par tous les algorithmes de coloration gloutonne. En effet, au moment où un sommet est colorié, il a au plus $\Delta(G)$ voisins déjà coloriés, il a donc au plus $\Delta(G)$ couleurs interdites. La plus petite couleur disponible est donc au plus $\Delta(G)$. Comme le graphe est simple, $\Delta(G) \leq n-1$, donc $\Delta(G) + 1 \leq n$: les couleurs utilisées sont toujours comprises entre 0 et $n-1$, et n colonnes suffisent pour `couleur_interdite`.

Pourquoi un tableau $n \times n$? La consigne impose d'utiliser uniquement des structures simples (tableaux, listes), sans table de hachage ni dictionnaire. Le tableau `couleur_interdite` permet de savoir en $O(1)$ si

une couleur est déjà utilisée par un voisin d'un sommet, simplement par un accès `couleur_interdite[s][c]`. Avec une liste des couleurs des voisins, ce test coûterait $O(n)$. Sa taille de $n \times n$ booléens reste en $O(n^2)$, ce qui respecte la complexité spatiale demandée.

3. Pseudocode

Données : un graphe simple et non orienté $G = (V, E)$ comptant $n \geq 1$ sommets (numérotés de 0 à $n-1$) et $m \geq 0$ arêtes, stocké dans `voisins`, un tableau de n listes d'adjacence.

Résultat : le tableau `couleur` de taille n , où `couleur[s]` est la couleur affectée au sommet s . C'est une coloration compatible, et les couleurs sont des entiers consécutifs à partir de 0.

```
1: Procédure DSATUR(voisins, n)

2:   // ----- Initialisation -----
3:   Pour sommet de 0 à n-1 faire
4:     degre[sommet]      <- taille de voisins[sommet]
5:     saturation[sommet] <- 0
6:     couleur[sommet]    <- -1
7:     Pour c de 0 à n-1 faire
8:       couleur_interdite[sommet][c] <- FAUX
9:     Fin pour
10:  Fin pour

11:  // ----- Boucle principale : un sommet colorié par itération -----
12:  Pour iteration de 1 à n faire

13:    // Choix du sommet à colorier :
14:    //   plus grand degré de saturation,
15:    //   puis plus grand degré initial,
16:    //   puis premier rencontré
17:    sommet_choisi <- -1
18:    Pour sommet de 0 à n-1 faire
19:      Si couleur[sommet] = -1 alors
20:        Si sommet_choisi = -1
21:          ou saturation[sommet] > saturation[sommet_choisi]
22:          ou (saturation[sommet] = saturation[sommet_choisi]
23:            et degre[sommet] > degre[sommet_choisi]) alors
24:            sommet_choisi <- sommet
25:        Fin si
26:      Fin si
27:    Fin pour

28:    // Plus petite couleur disponible pour sommet_choisi
29:    couleur_candidate <- 0
30:    Tant que couleur_interdite[sommet_choisi][couleur_candidate] = VRAI faire
31:      couleur_candidate <- couleur_candidate + 1
32:    Fin tant que
33:    couleur[sommet_choisi] <- couleur_candidate

34:    // Mise à jour des voisins non coloriés
35:    Pour chaque voisin de voisins[sommet_choisi] faire
36:      Si couleur[voisin] = -1
37:        et couleur_interdite[voisin][couleur_candidate] = FAUX alors
38:          couleur_interdite[voisin][couleur_candidate] <- VRAI
39:          saturation[voisin] <- saturation[voisin] + 1
40:      Fin si
41:    Fin pour

42:  Fin pour

43:  Retourner couleur
44: Fin Procédure
```

Remarque sur le premier sommet. L'algorithme 1 de la consigne colorie d'abord séparément le sommet de plus grand degré avec la couleur 0 (lignes 4 et 5), puis entre dans la boucle. Notre pseudocode traite ce cas directement dans la boucle principale, sans étape à part. À la première itération, aucun sommet n'est colorié et toutes les saturations valent 0: la règle de départage choisit donc le sommet de plus grand degré initial (le premier rencontré en cas d'égalité). Aucune couleur ne lui est interdite, il reçoit donc la couleur 0, puis ses voisins sont mis à jour. C'est exactement le comportement des lignes 4 et 5 de l'algorithme 1.

Remarque sur les couleurs consécutives. Un sommet ne reçoit la couleur c que si toutes les couleurs $0, \dots, c-1$ lui sont interdites, c'est-à-dire déjà utilisées par ses voisins. Les couleurs utilisées sont donc toujours consécutives à partir de 0.

4. Analyse de complexité temporelle

4.1 Initialisation

L'initialisation est une boucle **Pour sommet de 0 à $n-1$ faire** qui est exécutée exactement n fois. À chaque itération, les opérations suivantes sont effectuées:

- `degré[sommet] <- taille voisins[sommet]`, les voisins sont passés en paramètre à l'algorithme, sa taille est connue, on y a donc accès en $O(1)$.
- `saturation[sommet] <- 0` et `couleur[sommet] <- -1` sont deux affectations en $O(1)$.
- Il y a une boucle interne **Pour c de 0 à $n-1$ faire** qui fait n affectations `couleur_interdite[sommet][c] <- FAUX`. Son coût est donc de $O(n)$ pour chaque sommet.

Chaque itération de la boucle externe coûte $O(n)$. Cette boucle est répétée n fois, le coût total de l'initialisation est: $O(n) \times O(n) = O(n^2)$.

4.2 Boucle principale (n itérations)

4.2.1 Choix du sommet

À chaque itération de la boucle principale on doit sélectionner le sommet non colorié de plus grand degré de saturation. En cas d'égalité, on choisit celui de plus grand degré initial. S'il y a de nouveau égalité, on prend le premier sommet rencontré.

On parcourt tous les sommets avec l'aide de cette boucle **Pour sommet de 0 à $n-1$ faire**. Pour chaque sommet on fait:

- Un test `couleur[sommet] == -1` se fait en $O(1)$.
- Si le sommet est non colorié, une comparaison entre `saturation[sommet]` et `saturation[sommet_choisi]` se fait en $O(1)$.
- En cas d'égalité des saturations, une comparaison entre `degré[sommet]` et `degré[sommet_choisi]` se fait en $O(1)$.
- Si le sommet est meilleur, l'affectation `sommet_choisi <- sommet` se fait en $O(1)$.

Il n'y a pas de structure complexe, on parcourt des tableaux. Le coût d'une sélection est donc en $O(n)$. La boucle principale comporte n itérations, une par sommet à colorier, le coût total de toutes les sélections vaut: $O(n) \times O(n) = O(n^2)$.

4.2.2 Recherche de la plus petite couleur disponible

Une fois le sommet choisi, on doit lui attribuer la plus petite couleur qui n'est utilisée par aucun de ses voisins déjà coloriés.

On part de `couleur_candidate <- 0` et on l'incrmente avec la boucle **Tant que** `couleur_interdite[sommet_choisi][couleur_candidate] = VRAI` faire. À chaque tour on fait:

- Un accès au tableau `couleur_interdite[sommet_choisi][couleur_candidate]` qui se fait en $O(1)$.
- Une incrémentation `couleur_candidate <- couleur_candidate + 1` en $O(1)$.

Le sommet choisi a au plus $\deg(\text{sommet_choisi}) \leq n-1$ voisins, il a donc au plus $n-1$ couleurs interdites. Parmi les couleurs $0, 1, \dots, n-1$, au moins une est forcément libre. La boucle s'arrête donc après au plus n tests, et `couleur_candidate` reste toujours inférieure à n , on ne sort jamais du tableau `couleur_interdite`. L'affectation finale `couleur[sommet_choisi] <- couleur_candidate` se fait en $O(1)$.

Le coût d'une recherche est donc en $O(n)$. Elle est faite une fois par itération de la boucle principale, soit n fois, le coût total vaut: $O(n) \times O(n) = O(n^2)$.

4.2.3 Mise à jour des voisins

Après avoir colorié `sommet_choisi`, on doit mettre à jour les degrés de saturation de ses voisins non coloriés.

On parcourt la liste d'adjacence du sommet choisi avec la boucle **Pour chaque voisin de** `voisins[sommet_choisi]` faire. Pour chaque voisin on fait:

- Un test `couleur[voisin] = -1` qui se fait en $O(1)$.
- Un test `couleur_interdite[voisin][couleur_candidate] = FAUX` qui se fait en $O(1)$. Il permet de n'augmenter la saturation que si la couleur est nouvelle pour ce voisin, car le degré de saturation compte le nombre de couleurs **différentes** autour d'un sommet.
- Si les deux conditions sont vraies, deux affectations `couleur_interdite[voisin][couleur_candidate] <- VRAI` et `saturation[voisin] <- saturation[voisin] + 1` en $O(1)$.

Le coût d'une mise à jour est donc en $O(\deg(\text{sommet_choisi}))$. Comme chaque sommet est colorié exactement une fois, chaque liste `voisins[s]` n'est parcourue qu'une seule fois sur toute l'exécution. Le coût total vaut donc la somme des degrés: $\deg(0) + \deg(1) + \dots + \deg(n-1) = 2m$, soit $O(m)$. Comme le graphe est simple, $m \leq n(n-1)/2$, le coût total de toutes les mises à jour est donc en $O(m) \subseteq O(n^2)$.

4.3 Bilan

En additionnant le coût de chaque partie de l'algorithme:

Partie	Coût total
Initialisation (4.1)	$O(n^2)$

Partie	Coût total
Choix des sommets (4.2.1)	$O(n^2)$
Recherche des plus petites couleurs disponibles (4.2.2)	$O(n^2)$
Mises à jour des voisins (4.2.3)	$O(m)$
Retour du tableau <code>couleur</code>	$O(1)$

La complexité temporelle totale vaut donc: $O(n^2) + O(n^2) + O(n^2) + O(m) = O(n^2 + m)$. Comme le graphe est simple, $m \leq n(n-1)/2$, donc $O(m) \subseteq O(n^2)$. La complexité temporelle de l'algorithme DSATUR est de $O(n^2)$ dans le pire des cas.

5. Analyse de complexité spatiale

L'analyse de complexités spatiales consiste à évaluer la quantité de mémoire utilisée par notre algorithme en fonction de la taille d'entrée. On distingue l'espace occupé par les structures de données auxiliaires de celui occupé par l'entrée elle-même.

Les structures de données utilisées:

- `degré` est un tableau d'entiers de taille n , donc $O(n)$.
- `saturation` est un tableau d'entiers de taille n , donc $O(n)$.
- `couleur` est un tableau d'entiers de taille n , donc $O(n)$.
- `couleur_interdite` est un tableau de boolean de taille $n \times n$, donc $O(n^2)$.
- Il y a des variables comme `sommet` ou `sommet_choisi`, ce sont des entiers ou des booléens, donc en $O(1)$.
- `voisins` est le tableau donné en entrée. C'est un tableau de n listes d'adjacence. La somme des tailles de ces listes est $2m$. L'espace occupé par l'entrée est en $O(n + m)$.

La complexité spatiale est dominée par le tableau `couleur_interdite` et vaut $O(n^2)$. En incluant l'entrée, on ajoute $O(n + m)$, mais on reste en $O(n^2)$ car $m \leq n(n-1)/2$. Dans les deux cas, la complexité spatiale totale de l'algorithme DSATUR est de $O(n^2)$ dans le pire des cas.

6. Conclusion

Dans ce travail, nous avons proposé un pseudocode détaillé et complet de l'heuristique DSATUR. Il prend en entrée un graphe simple et non orienté stocké sous forme de tableau de listes d'adjacence, et retourne un tableau contenant la couleur de chaque sommet, les couleurs étant des entiers consécutifs à partir de 0.

Notre implémentation utilise uniquement des structures simples: des tableaux d'entiers de taille n , un tableau de booléens de taille $n \times n$ et les listes d'adjacence données en entrée. Aucune structure complexe (table de hachage, dictionnaire, tas indexé) n'est utilisée.

L'analyse montre que:

- la complexité temporelle est en $O(n^2)$ dans le pire des cas, car chacune des n itérations de la boucle principale coûte $O(n)$, en plus d'une initialisation en $O(n^2)$ et de mises à jour en $O(m)$ au total;

- la complexité spatiale est en $O(n^2)$ dans le pire des cas, dominée par le tableau `couleur_interdite`.

Les deux complexités demandées par la consigne sont donc bien respectées.